

Maven Overview:

Maven is a powerful build and project management tool, introduced by Apache, and is widely used in Java-based projects. It helps developers manage dependencies, build artifacts like `.war` and `.jar` files, and automate the build process. It uses a `pom.xml` file, which contains configuration details like project dependencies, plugins, goals, and more. Maven supports various platforms, including Windows, Mac, and Linux, and is compatible with multiple project structures.

Maven and Artifact Creation:

When you build code using Maven, it compiles your project and bundles the code into artifacts, such as `.war` or `.jar` files, depending on the project type. These artifacts are stored in the `target` directory. After creating the artifact, you can deploy it to a web server, such as Apache Tomcat, in the case of web applications.

Maven works on a standard directory structure and uses `pom.xml` (Project Object Model) for building and managing dependencies. If your team uses other tools like Gradle, it's similar in functionality, and it's advisable to request a knowledge transfer (KT) from your team lead to better understand their toolchain.

Installation of Java and Maven on Windows:

Before installing Maven, you need Java installed on your system. Here are the steps for installing Java and Maven on Windows:

1. Install Java:

- It's recommended to use a long-term support (LTS) version like Java 1.8 (also known as Java 8).
- Search for "Java 1.8 download" and download it from Oracle's official website. You will need to create an Oracle account to download Java.
- You will find two components:
 - **JDK (Java Development Kit):** For writing, compiling, and running Java code.
 - **JRE (Java Runtime Environment):** Only required to run Java applications.
- Once downloaded and installed, Java files will be placed in `C:\Program Files\Java`.

2. Setting Up Environment Variables for Java:

- After installation, configure the `JAVA_HOME` environment variable:
 - Right-click on **This PC** or **My Computer** and select **Properties**.
 - Click on **Advanced system settings** and then **Environment Variables**.

- Under **User Variables**, click **New** and add **JAVA_HOME** as the variable name and the path to your JDK installation as the value (e.g., **C:\Program Files\Java\jdk1.8.x**).
- Do the same under **System Variables**.

To verify, open the command prompt and run:

```
java -version
```

- If the version is displayed, Java is successfully installed.

3. Install Maven:

- Search for "Install Maven" on Google, and download the latest stable version from the official Apache Maven website.
- Download the ZIP file, extract it, and move it to your preferred directory.

4. Setting Up Environment Variables for Maven:

- Configure the **MAVEN_HOME** environment variable just like you did for Java:
 - Under **User Variables**, add **MAVEN_HOME** and the path to your Maven installation.
 - Also, add the Maven **bin** directory to the system **PATH** (e.g., **C:\Program Files\Apache\maven\bin**).

Verify the installation by running:

```
mvn -version
```

- If the version is displayed, Maven is successfully installed.

Maven Repositories:

Maven uses three types of repositories to manage dependencies and plugins:

1. **Local Repository:** Located on your system in the **.m2** directory (usually under **C:\Users\YourName\.m2\repository**). When you build a Maven project, it first checks the local repository for dependencies.
2. **Central Repository:** If the required dependencies are not found locally, Maven checks the central repository maintained by Apache.
3. **Remote Repository:** Some projects may also have remote repositories configured, where Maven can pull dependencies from.

When you run a Maven build command like `mvn clean install`, Maven checks these repositories in the following order: local → central → remote. If dependencies are found in the local repository, it uses them; otherwise, it downloads them from the central or remote repositories.

Key Maven Commands:

- **mvn clean install:** This command cleans the project, compiles the code, runs tests, and generates the artifact (usually a `.jar` or `.war` file).
- **mvn archetype:generate:** This command helps create a new project by providing an archetype (template) and generates a basic directory structure along with a `pom.xml` file.

Maven Archetype Project Generation:

When you run the `mvn archetype:generate` command, Maven will prompt you to choose the type of project and version you want to generate. You'll also provide the following details:

- **Group ID:** Represents the organization or group the project belongs to.
- **Artifact ID:** The unique name of the project.
- **Version:** Maven will often append `SNAPSHOT` to the version, indicating that the project is under development.

Once the directory structure is generated, Maven will also create a basic `pom.xml` file, which serves as the foundation for building and managing the project. Developers can customize the `pom.xml` file according to the project's requirements.